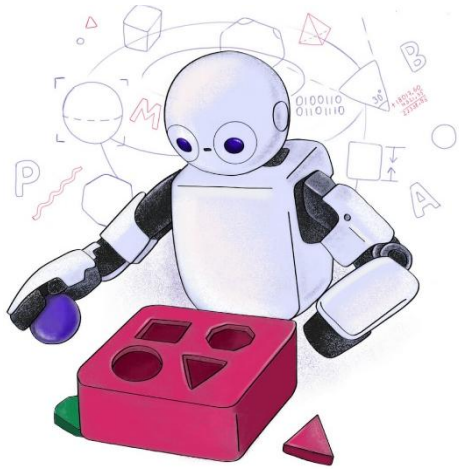
T319 - Introdução ao Aprendizado de Máquina: *Regressão Linear (Parte IV)*



Inatel

Felipe Augusto Pereira de Figueiredo
felipe.figueiredo@inatel.br

Recapitulando

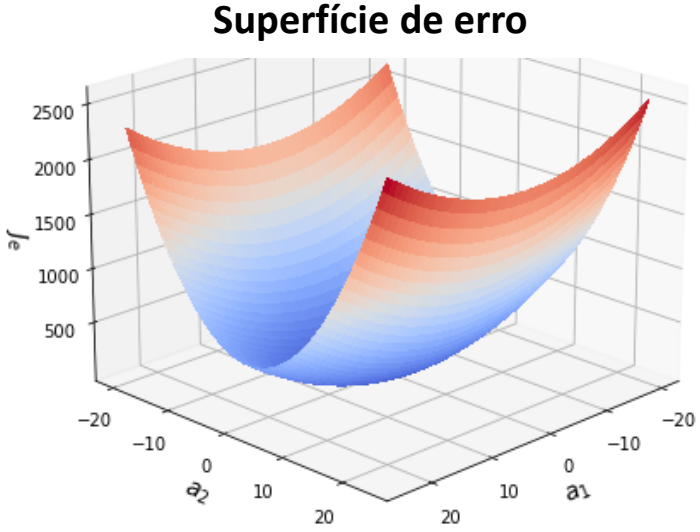
- Vimos que o ***valor da taxa de aprendizagem*** ***influencia no processo de aprendizagem*** do gradiente descendente.
  - Valores pequenos fazem com que o algoritmo tenha convergência muito lenta.
  - Valores muito grandes fazem com que o algoritmo divirja.
- O gráfico do erro versus iterações nos ajuda a ***depurar as versões do GD***.
- Quando usamos as versões estocásticas do GD, podemos ***reduzir o valor da taxa de aprendizagem ao longo do treinamento*** para "***forçar***" a ***convergência*** do algoritmo.
- Nesse tópico, veremos
  - Técnicas de ***pré-processamento*** importantes para algoritmos de ML que usam ***métricas de distância como função de erro***.
  - Como ***polinômios*** podem ser usados para ***se ajustar a dados que apresentam mapeamento não-linear*** entre os atributos e o valor esperado.

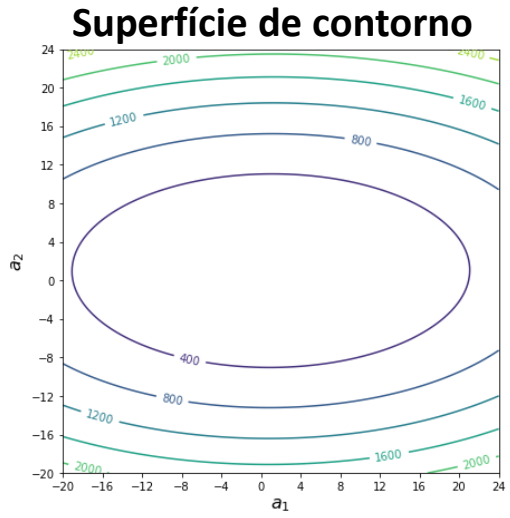
Variações do formato da superfície de erro

- Como visto no laboratório #3, nem toda superfície de erro criada a partir da função do EQM tem formato de tigela (i.e., com curvas de erro circulares).
- Dependendo dos ***intervalos de variação dos atributos***, podemos ter ***superfícies com formato de vale***.
- Por exemplo, na função hipótese abaixo, se $x_1 \gg x_2$, ele ***dominará o erro*** e fará com que a superfície de erro tenha ***formato de vale***.

$$J_e(\boldsymbol{a}) = \frac{1}{N}\sum_{n=0}^{N-1}\left[y_{\text{noisy}}(n) - \underbrace{\left(\hat{a}_1 x_1(n) + \hat{a}_2 x_2(n)\right)}_{\text{Função hipótese}}\right]^2 \underset{x_1 \gg x_2}{\approx} \frac{1}{N}\sum_{n=0}^{N-1}\left[y_{\text{noisy}}(n) - \hat{a}_1 x_1(n)\right]^2$$

Superfícies com formato de vale

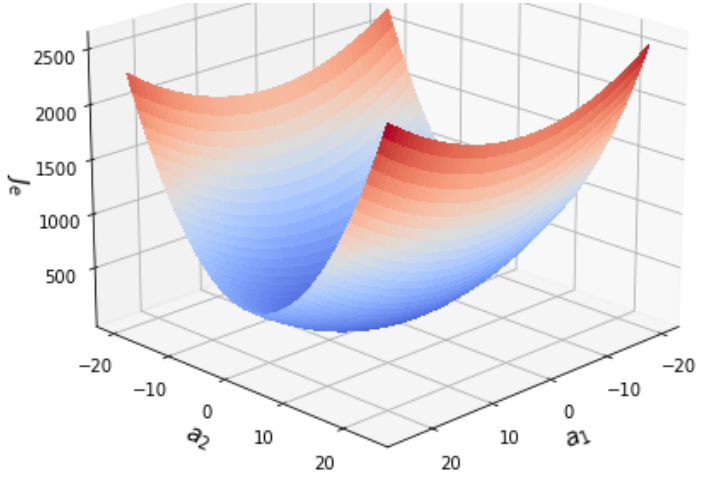




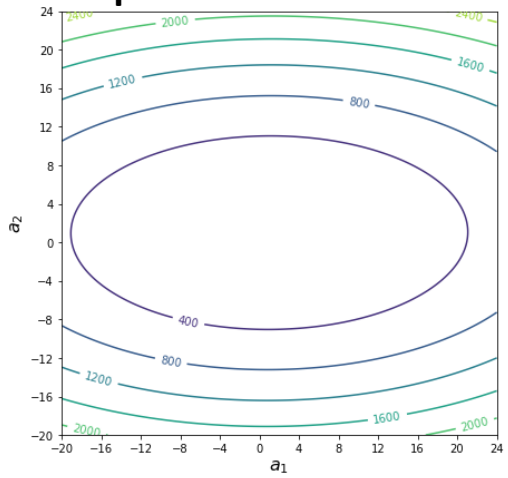
- Superfícies com ***formato de vale*** podem fazer com que a ***convergência do GD se torne muito lenta***.
- A ***convergência*** se torna ***lenta devido à superfície ser plana ou quase plana*** em algumas direções.
  - Se em uma determinada direção a inclinação é ≈ 0, a derivada parcial tende a zero.

Superfícies com formato de vale

Superfície de erro



Superfície de contorno



- Nessas direções, ***as derivadas parciais da função de erro são muito pequenas***, tornando as ***atualizações dos pesos***, consequentemente, ***muito pequenas***.
- Na figura ao lado, as derivadas parciais do EQM em relação ao peso a_1 serão muito pequenas devido à ***pequena inclinação da superfície nessa direção***.

$$a_1 \leftarrow a_1 - \alpha \frac{\partial J_e(\boldsymbol{a})}{\partial a_1}$$

O que pode ser feito?

Escalonamento de atributos

- Para evitar esse problema, o ***intervalo de variação de todos os atributos*** pode ser ***escalonado***, trazendo-os para uma escala similar.
- Assim, ***cada atributo influenciará da mesma forma o cálculo do erro***.

$$J_e(\boldsymbol{a}) = \frac{1}{N}\sum_{n=0}^{N-1}\left[y_{\text{noisy}}(n) - \underbrace{\left(\hat{a}_1 x_1(n) + \hat{a}_2 x_2(n)\right)}_{\text{Função hipótese}}\right]^2 .$$

- Consequentemente, a superfície se tornará mais circular, facilitando o aprendizado.
- As duas formas mais comuns de escalonamento são:
  - **Normalização Mín-Max**
  - **Padronização**

Escalonamento de atributos

Observações:

- Em geral, aplicamos o escalonamento ***apenas aos atributos*** e não aos rótulos, pois ***são os atributos que influenciam o formato da superfície*** de erro.
- Além disso, ao escalonar os rótulos, perde-se seu significado.
  - Por exemplo, a predição do preço de casas deixa de ser em reais.
- O escalonamento (qualquer tipo) altera os valores dos pesos originais.
  - Se a escala dos atributos é alterada, para que o modelo gere valores de saída no intervalo original dos rótulos, os pesos precisam ter seus valores alterados (ver anexo I).
- O escalonamento pode ser usado com qualquer versão do GD.

Escalonamento de atributos

- Em geral, a ***normalização mín-max*** faz com que os ***atributos variem entre 0 e 1***, mas pode-se definir outros intervalos.
- A equação usada para normalizar os atributos é apresentada abaixo

$$x'_k(n) = \frac{x_k(n) - \min(\boldsymbol{x}_k)}{\max(\boldsymbol{x}_k) - \min(\boldsymbol{x}_k)}, 0 \leq x'_k(n) \leq 1,$$

onde x_k representa o k–ésimo atributo, n é o número da amostra, $\min(\boldsymbol{x}_k)$ e $\max(\boldsymbol{x}_k)$ são os valores mínimo e máximo, respectivamente, ***calculados ao longo de todas as amostras do k–ésimo vetor de atributos***, $\boldsymbol{x}_k$.

Escalonamento de atributos

- ***Padronização*** faz com que os ***atributos passem a ter média zero e desvio padrão unitário***.
- Notem que, neste caso, os ***valores não ficam restritos a um intervalo específico***.
- A equação usada para padronizar os atributos é apresentada abaixo.

$$x'_k(n) = \frac{x_k(n) - \mu_{x_k}}{\sigma_{x_k}},$$

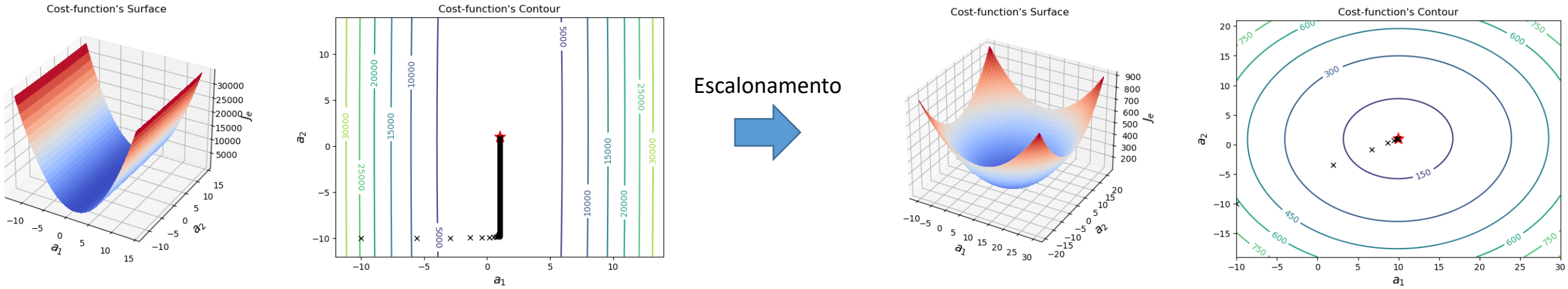
onde x_k representa o k–ésimo atributo, n é o número da amostra, μ_{x_k} e σ_{x_k} são as ***estimativas*** da média e do desvio padrão, respectivamente, ***calculados ao longo de todas as amostras do k–ésimo vetor de atributos***, $\boldsymbol{x}_k$.

Exemplo: escalonamento_de_atributos_com_scikit_learn.ipynb

Exemplo: padronização_de_atributos_com_scikit_learn.ipynb

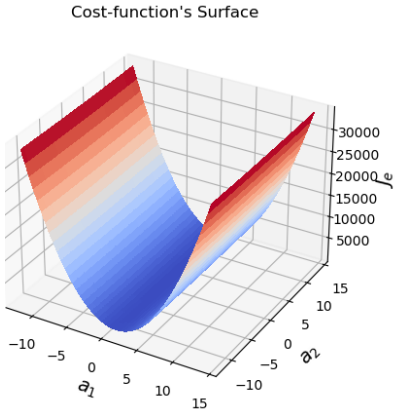
Vantagens do escalonamento de atributos

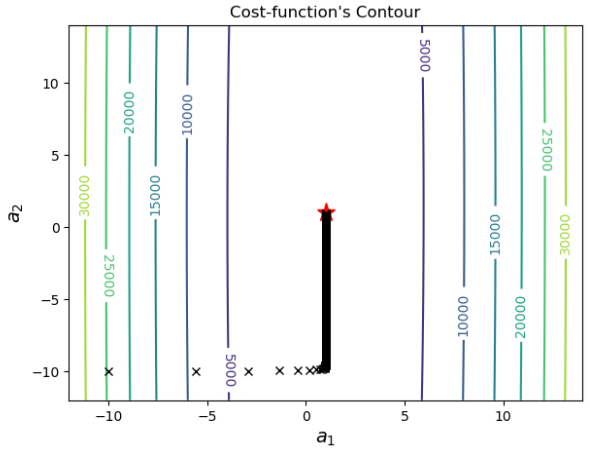
- ***Acelera a convergência do gradiente descendente***: superfície de erro fica mais circular e o gradiente passa a apontar direto para o mínimo.
  - Faz com que a inclinação da superfície se torne similar em todas as direções.
- ***Melhora a estabilidade numérica***: atributos com escalas diferentes geram gradientes de magnitudes muito diferentes por peso.
  - Sem escalonamento, os valores absolutos dos ***gradientes*** podem se tornar ***extremamente grandes*** e podem não ser representados pelas variáveis.



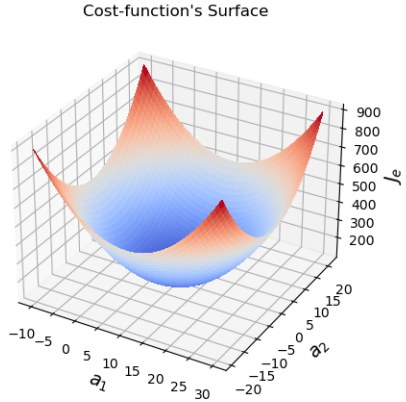
Vantagens do escalonamento de atributos

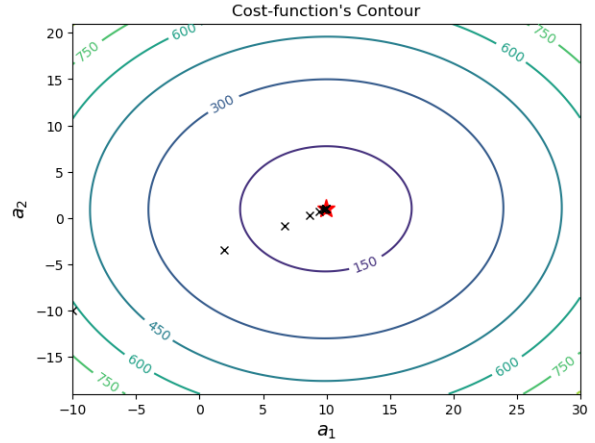
- Permite ***comparar a influência*** de cada ***atributo*** no modelo de forma justa.
  - Pois os pesos representam o ***impacto dos atributos*** nas predições (importância para o modelo).
- Evita que ***atributos com escalas muito diferentes dominem o processo de treinamento***.
  - O gradiente é dominado por eles, o que exige uma taxa de aprendizado pequena e retarda o aprendizado dos demais pesos.





Escalonamento

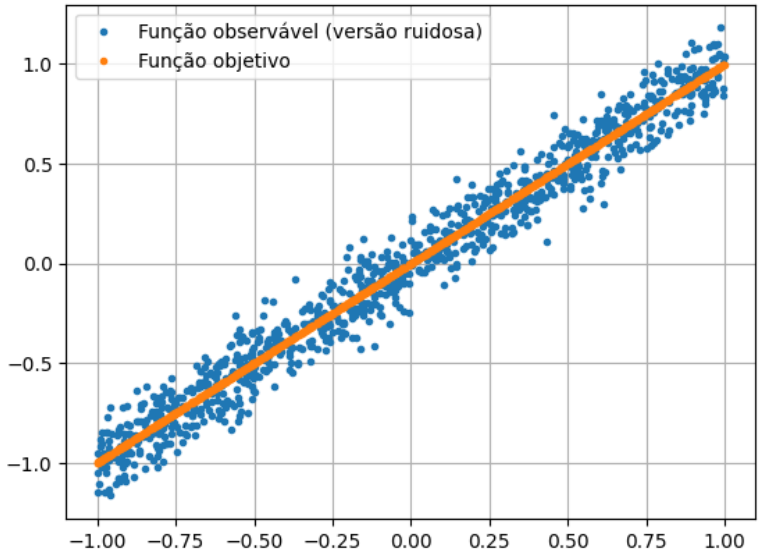


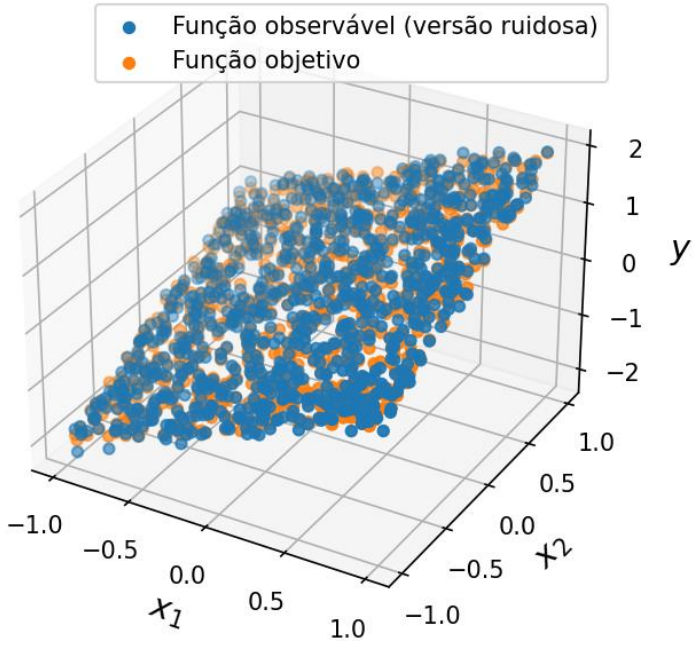


Avisos

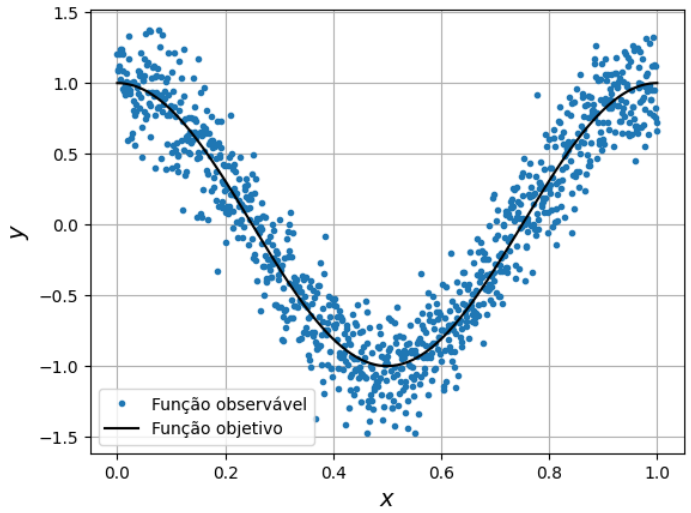
- **Exercício Prático**: **Laboratório #5** (**Apenas o exercício #1**)
  - Pode ser acessado através do link acima (Google Colab) ou no GitHub.
  - Vídeo explicando o laboratório: Arquivos -> Material de Aula -> Laboratório #5
  - Se atentem aos prazos de entrega.
  - Instruções para resolução e entrega dos laboratórios.
- **Avaliação Presencial**: **06/11/2026 – Sala I-20**
  - Avaliação e projeto podem ser feitos em grupos de no máximo 3 alunos.
  - **Presencialmente, faremos apenas o exercício 1 do projeto final.**
  - Os outros dois exercícios devem ser entregues até **27/11/2026**.

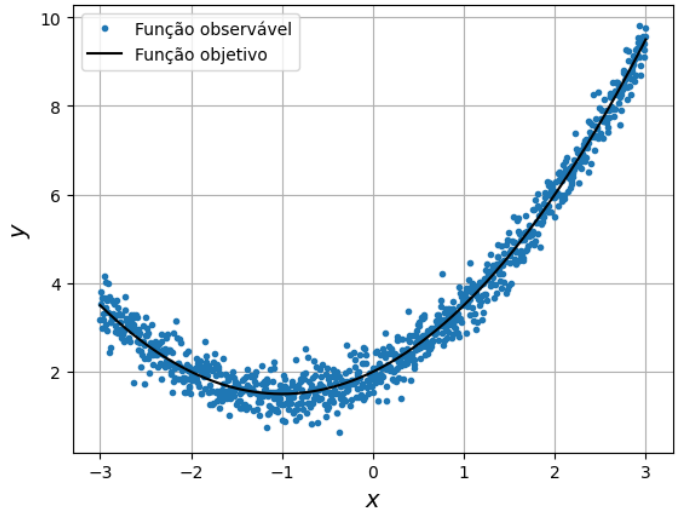
Até agora, usamos ***funções hipóteses com formato de hiperplanos***, e.g., retas e planos, para aproximar ***mapeamentos lineares*** entre os atributos e o valor esperado.

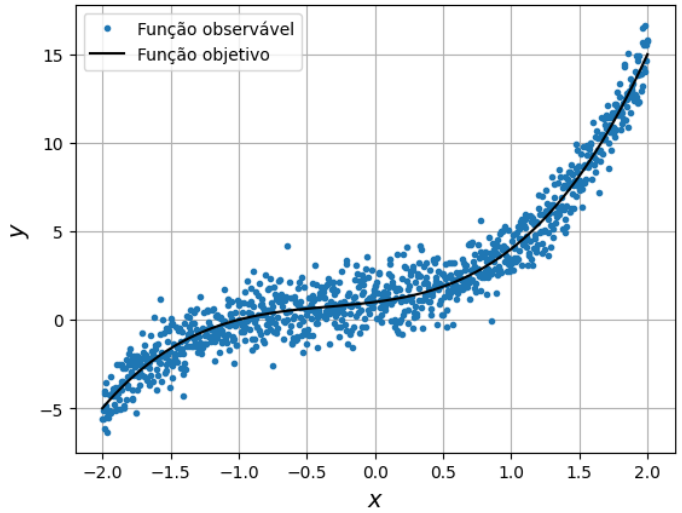




Mas e se o mapeamento entre os atributos e o valor esperado for não linear?



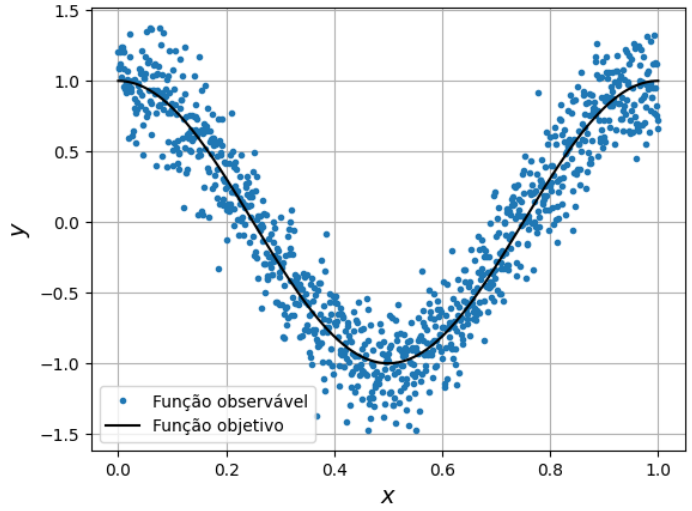


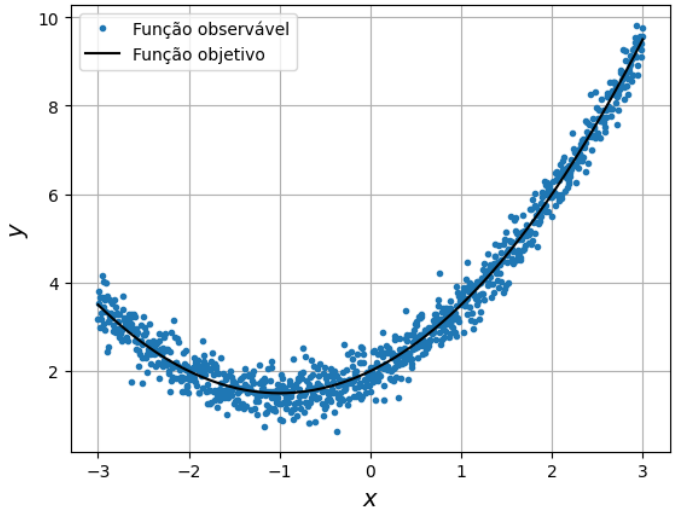


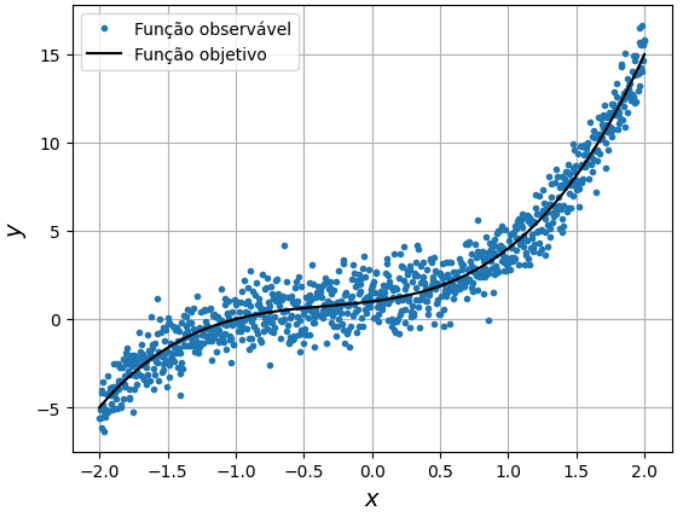
O que podemos fazer quando ***hiperplanos não se ajustam bem ao comportamento geral dos dados***?

Mapeamentos não lineares

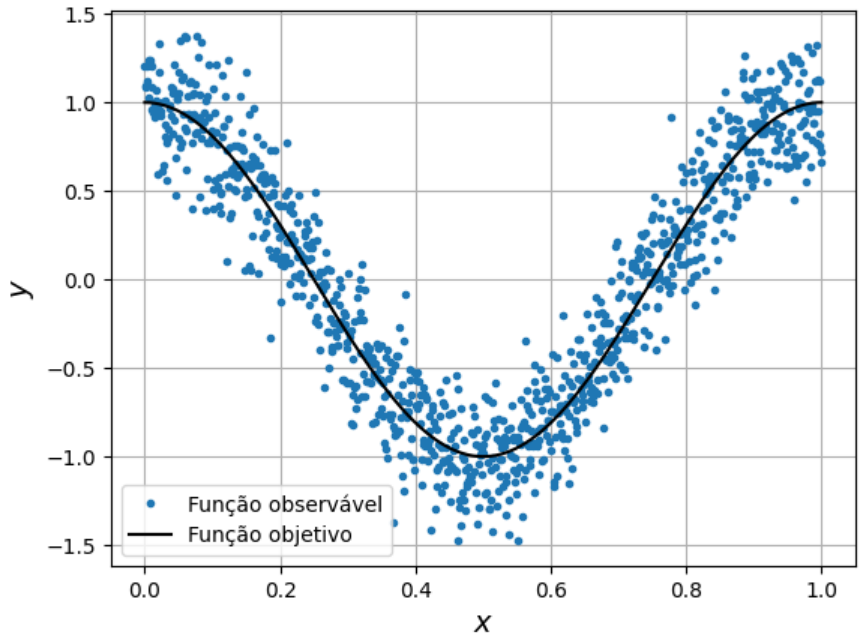
- Observem as figuras abaixo, uma ***reta*** claramente ***não seria uma boa escolha para aproximar esses mapeamentos não lineares***.
  - ***Retas não capturariam o comportamento das funções abaixo***, pois elas não têm ***flexibilidade*** (i.e., graus de liberdade ou complexidade) o suficiente para isso.
- Portanto, qual tipo de função hipótese seria mais apropriado para aproximar esses comportamentos não lineares?







Regressão polinomial



- O teorema da aproximação de ***Stone-Weierstrass*** diz que *"qualquer função contínua no intervalo fechado [a, b] pode ser uniformemente aproximada por um* ***polinômio****"*.
- Portanto, podemos aproximar ***qualquer tipo de mapeamento (linear ou não linear)*** com ***polinômios***, bastando apenas ***encontrar o grau ideal*** e, claro, os pesos do polinômio.

Regressão polinomial

- Exemplo de um polinômio de grau 4* com três atributos, x_1, x_2 e x_3

$$y(x_1, x_2, x_3) = a_0 + a_1x_1 + a_2x_2 + a_3x_3 + a_4x_1x_3 + a_5{x_1}^3 + a_6{x_1x_2x_3}^2$$

- *O grau do polinômio é o ***maior valor*** resultante da ***soma dos expoentes dos atributos*** de um monômio.
- Percebam que alguns monômios são formandos pela ***combinação dos atributos originais***, formando ***novos atributos***.
- **OBS**.: Um problema com polinômios é que a presença de ***outliers*** nos dados de treinamento impacta o desempenho do modelo.
  - Por exemplo, um *outlier* do atributo x_1 com valor igual a 100, no sexto monômio, ${x_1}^3$, se torna 1.000.000.

Regressão polinomial

- Por simplicidade didática, inicialmente, nós consideraremos ***funções hipóteses polinomiais em uma variável*** (i.e., com um atributo):

$$h(x_1(n)) = a_0 + a_1 x_1(n) + a_2 {x_1}^2(n) + \cdots + a_M {x_1}^M(n) = \boldsymbol{a}^T \boldsymbol{x}(n),$$

onde $n = 1 \ldots N$ é o número da amostra, M é o grau do polinômio, $\boldsymbol{a} = [a_0 \ a_1 \ a_2 \ \ldots \ a_M]^T \in \mathbb{R}^{M+1\times 1}$, $\boldsymbol{x}(n) = [x_0 \ x_1(n) \ {x_1}^2(n) \ldots \ {x_1}^M(n)]^T \in \mathbb{R}^{M+1\times 1}$ e $x_0 = 1$ é o atributo de *bias,* associado ao peso de *bias*, a_0.

- ***Todos resultados encontrados anteriormente*** (equação normal, vetor gradiente para implementação de qualquer versão do algoritmo do gradiente descendente, escalonamento) ***são diretamente estendidos para funções hipótese polinomiais***.

Regressão polinomial

- Só precisamos nos lembrar que o ***vetor de atributos***, $\boldsymbol{x}$, e consequentemente, a ***matriz de atributos***, $\boldsymbol{X}$, usados na regressão devem ***conter os atributos dos monômios***, sejam eles os originais ou suas combinações.
- Por exemplo, para a seguinte função hipótese polinomial

$$h(x_1(n)) = a_0 + a_1 x_1(n) + a_2 {x_1}^2(n) + \cdots + a_M {x_1}^M(n),$$

a ***matriz de atributos polinomial***, $\boldsymbol{X}$, fica da seguinte forma

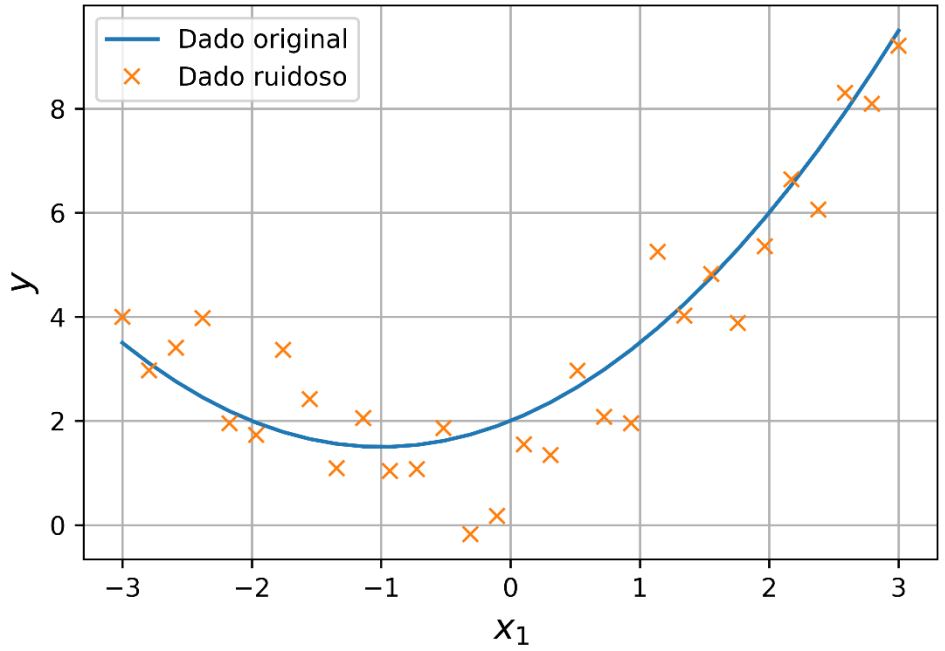
$$\boldsymbol{X} = \begin{bmatrix} 1 & x_1(0) & {x_1}^2(0) & \cdots & {x_1}^M(0) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1(N-1) & {x_1}^2(N-1) & \cdots & {x_1}^M(N-1) \end{bmatrix} \in \mathbb{R}^{N \times M+1},$$

onde cada coluna contém um atributo dos monômios.

* A biblioteca SciKit-Learn possui uma função que cria essas matrizes automaticamente a partir dos atributos originais.

Porém, o desafio agora é ***encontrar o grau do polinômio*** que melhor se aproxime os dados.

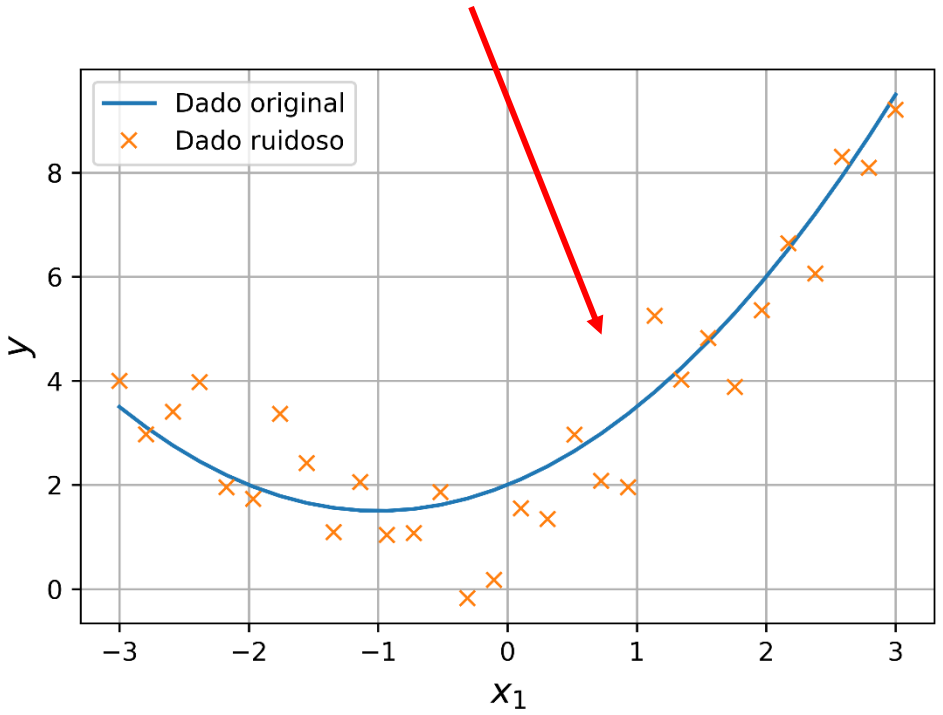
Exemplo de regressão usando polinômio



Usando apenas os ***dados ruidosos*** mostrados na figura acima, ***qual o grau e os pesos de um polinômio*** para que ele se aproxime da ***função objetivo*** da melhor forma possível?

Exemplo de regressão usando polinômio

Função objetivo: polinômio de ordem 2.



A partir do dados ruidosos, queremos encontrar um polinômio (pesos e ordem) que melhor se aproxime da função objetivo.

- Para exemplificar essa questão da busca pela ***melhor ordem*** do polinômio aproximador, geramos ***30 amostras*** da seguinte função objetivo (polinômio de grau 2)

$$y\big(x_1(n)\big) = 2 + x_1(n) + 0.5{x_1}^2(n),$$

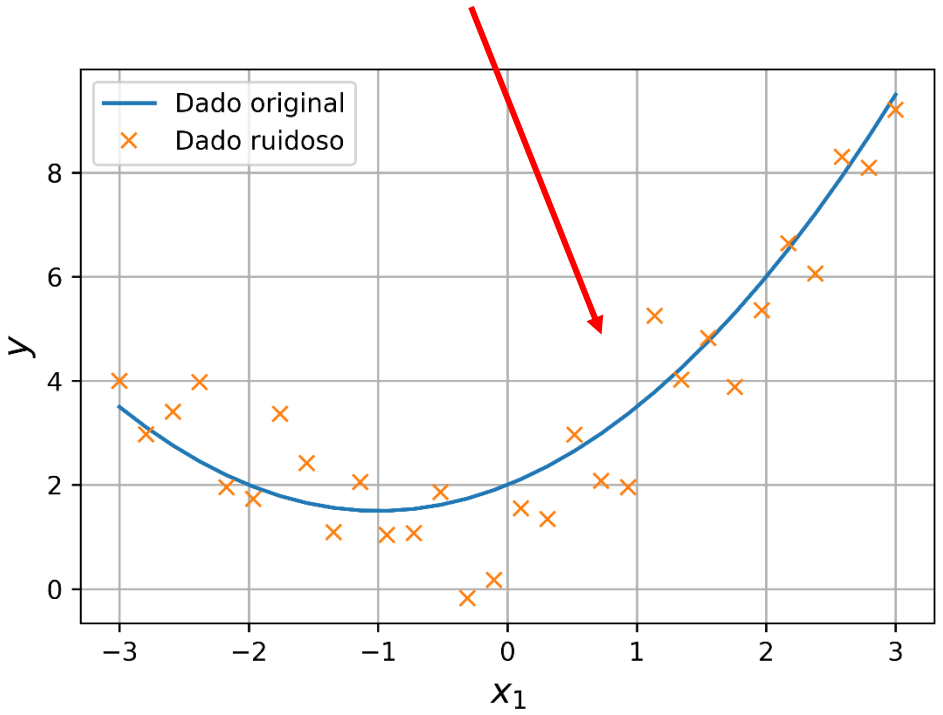
e adicionamos ruído Gaussiano branco, $w(n)$

$$y_{\text{noisy}}\big(x_1(n)\big) = y\big(x_1(n)\big) + w(n),$$

onde $x_1(n)$ são valores linearmente espaçados entre -3 e 3 e $w(n) \sim N(0,1)$.

Exemplo de regressão usando polinômio

Função objetivo: polinômio de ordem 2.



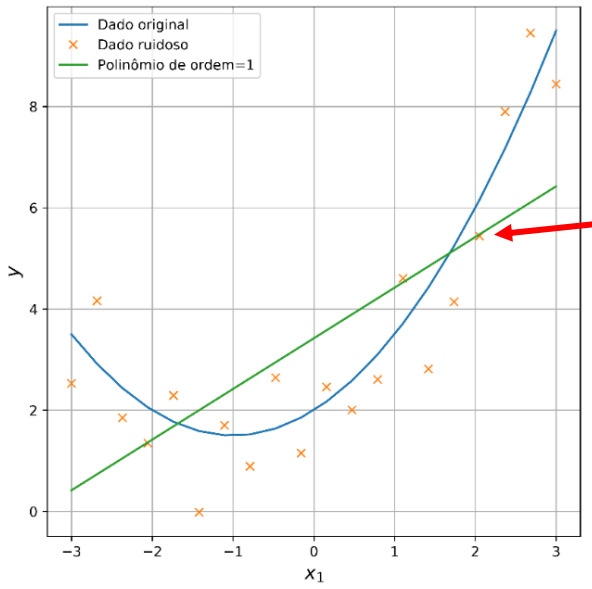
A partir do dados ruidosos, queremos encontrar um polinômio (pesos e grau) que melhor se aproxime da função objetivo.

- Vamos usar uma ***função hipótese polinomial*** para ***aproximar a função objetivo a partir dos dados ruidosos***.
- Porém, surge uma dúvida, ***e se não soubéssemos o grau por trás do modelo gerador, qual grau deveríamos utilizar***?

Exemplo: polynomial_regression.ipynb

Regressão polinomial: Qual ordem usar?

Polinômio de ordem 1.



Reta não é flexível o suficiente para se contorcer e aproximar o comportamento da função objetivo.

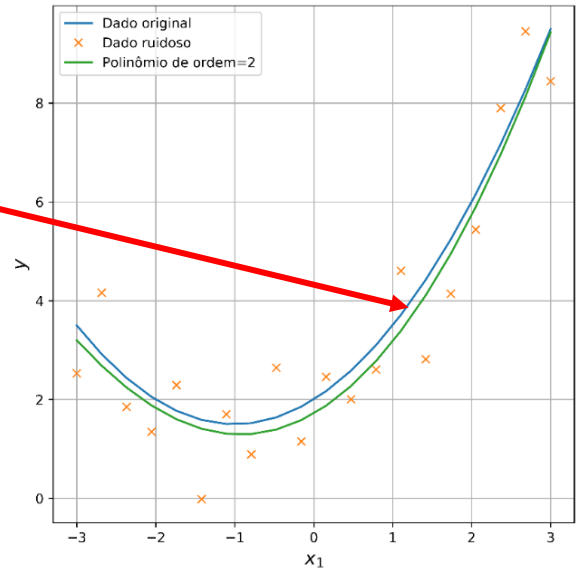
Flexibilidade e **grau de generalização** muito baixos.

- Polinômio de ordem 1 (i.e., reta) ***não tem flexibilidade o suficiente*** para aproximar o comportamento por trás das amostras ruidosas, i.e., a função objetivo.
- O erro (MSE) é alto para exemplos dos conjuntos de treinamento e de validação (i.e., exemplos não vistos durante o treinamento).
- Efeito conhecido como ***subajuste*** ou ***underfitting***: ***flexibilidade*** e ***grau de generalização*** muito baixos.

Regressão polinomial: Qual ordem usar?

Polinômio de ordem 2.

Ordem ótima pois é a mesma do modelo gerador.

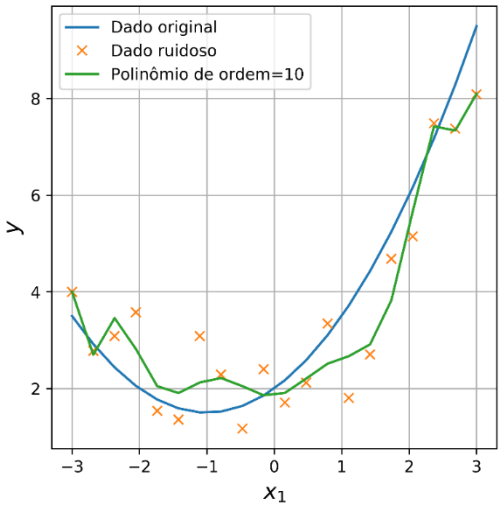


Encontra relação de compromisso entre **flexibilidade** e **generalização**: **flexibilidade** e **grau de generalização** médios.

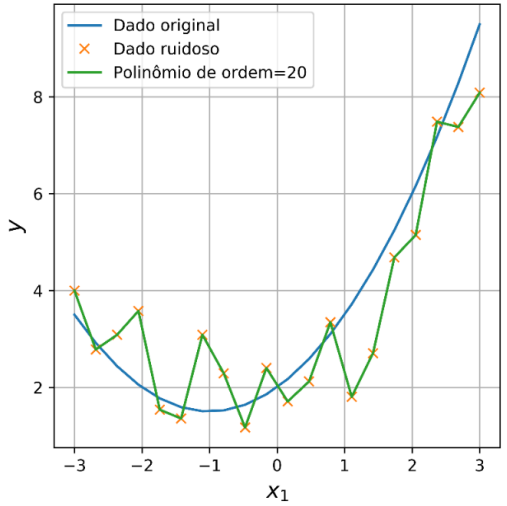
- Porém, como esperado, o polinômio de ***ordem 2*** produz a ***melhor aproximação*** da função objetivo, ***errando pouco para exemplos dos conjuntos de treinamento e validação***.
  - Esse modelo encontra uma ***relação de compromisso*** entre ***flexibilidade*** e ***grau de generalização***.
  - Essa aproximação será melhor quanto maior for o conjunto de treinamento e/ou menor o ruído.

Regressão polinomial: Qual ordem usar?

Polinômio de ordem 10.



Polinômio de ordem 20.



Polinômio de ordem 30.



- Polinômios com ordem ***maior do que 2*** tendem a produzir ***aproximações perfeitas*** dos exemplos disponíveis, i.e., o modelo ***memoriza o conjunto de treinamento***.
- O ***erro*** no ***conjunto de treinamento é muito baixo***.
- Porém, essa ***aproximação se distancia bastante do modelo gerador***.
- Portanto, esses modelos apresentarão ***erros significativamente maiores*** quando forem apresentados a ***exemplos de validação***.
- Efeito conhecido como ***sobreajuste*** ou ***overfitting***: ***flexibilidade*** muito alta e ***grau de generalização*** muito baixo.

Resumo sobre subajuste e sobreajuste

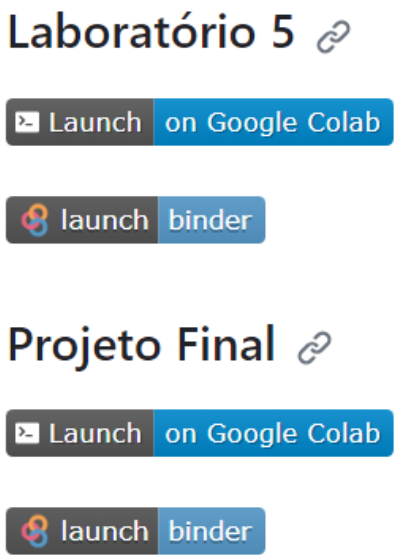
- **Subajuste**: situação em que o modelo ***falha em aproximar o comportamento geral*** por trás das amostras ***devido à falta de flexibilidade (ou capacidade)***.
  - Ocorre devido ao modelo não ter graus de liberdade suficientes para a aproximação.
  - O modelo produz ***erros significativos*** tanto quando apresentado ao próprio ***conjunto de treinamento quanto a dados inéditos***.
  - Se o modelo está subajustando, mesmo que o número de ***exemplos aumente indefinidamente***, ***esta situação não vai desaparecer***, é necessário ***aumentar a flexibilidade do modelo***, ou seja, no caso da regressão polinomial, sua ordem.

Resumo sobre subajuste e sobreajuste

- **Sobreajuste**: situação em que o ***modelo se ajusta tão bem aos exemplos de treinamento que ele aprende até o ruído*** presente nos mesmos (baixo ***erro de treinamento***).
- Porém, o modelo produz ***erros significativos quando apresentado a dados inéditos*** (alto erro de ***erro de validação***).
  - Ocorre devido ao alto grau de flexibilidade do modelo.
  - Se o modelo está sobreajustando, então é necessário ***diminuir sua flexibilidade*** ou ***aumentar o conjunto de treinamento*** até que o erro de validação atinja o erro de treinamento.
- Nosso ***objetivo*** será encontrar um modelo que apresente uma boa relação de compromisso entre ***flexibilidade*** e ***capacidade de generalização***.
  - Flexibilidade suficiente para capturar o comportamento geral e generalizar bem.

Tarefas

- **Quiz**: “*T319 - Quiz - Regressão: Parte IV*” que se encontra no MS Teams.
- **Exercício Prático**: **Laboratório #5**.
  - Pode ser acessado através do link acima (Google Colab) ou no GitHub.
  - Vídeo explicando o laboratório: Arquivos -> Recordings -> Laboratório #5
  - Se atentem aos prazos de entrega.
  - Instruções para resolução e entrega dos laboratórios.
- **Entrega do projeto final: 27/11/2026**
  - **Projeto final já se encontra no github.**
  - **Pode ser feito em grupos de no máximo 3 alunos.**



Obrigado!

Anexo I: O escalonamento altera o valor dos pesos originais

Mudança dos pesos originais após a padronização

- Considerando a seguinte função hipótese

$$\hat{y}(n) = \hat{a}_1 x_1(n).$$

- Se padronizarmos o atributo x_1, teremos

$$x'_1(n) = \frac{x_1(n) - \mu_{\boldsymbol{x}_1}}{\sigma_{\boldsymbol{x}_1}}, \forall n,$$

onde μ e σ são as estimativas da média e do desvio padrão, respectivamente, calculados ao longo de todas as amostras do vetor de atributos, $\boldsymbol{x}_1$.

Mudança dos pesos originais após a padronização

- Isolando-se $x_1(n)$ na equação da padronização, temos
$$x_1(n) = x_1'(n)\sigma_{\boldsymbol{x_1}} + \mu_{\boldsymbol{x_1}}.$$
- Na sequência, substituindo-se $x_1(n)$ na função hipótese, tem-se
$$\hat{y}(n) = \hat{a}_1\left(x_1'(n)\sigma_{\boldsymbol{x_1}} + \mu_{\boldsymbol{x_1}}\right) = \hat{a}_1\sigma_{\boldsymbol{x_1}}x_1'(n) + \hat{a}_1\mu_{\boldsymbol{x_1}}.$$
- Perceba que na equação acima há o surgimento de um termo de bias, $\hat{a}_1\mu_{\boldsymbol{x_1}}$, além da alteração do peso original para $\hat{a}_1\sigma_{\boldsymbol{x_1}}$.

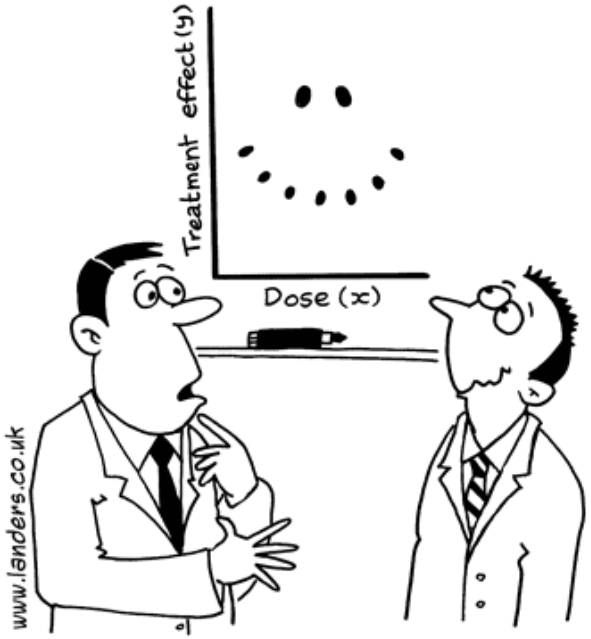
Mudança dos pesos originais após a padronização

- Assim, podemos reescrever a equação acima como

$$\hat{y}(n) = \hat{a}'_0 + \hat{a}'_1 x'_1(n),$$

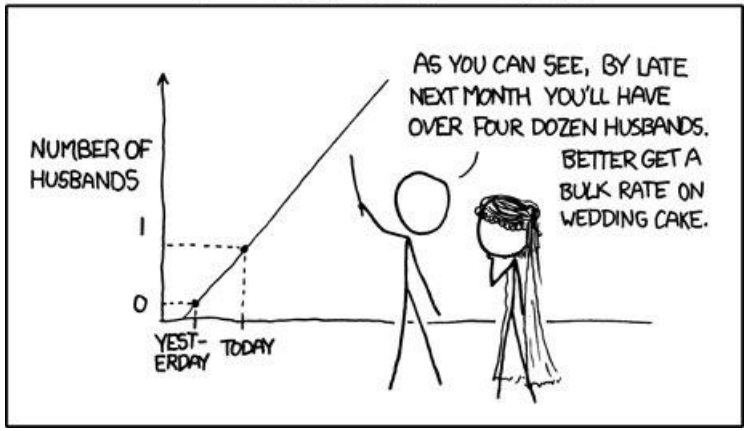
onde $\hat{a}'_0 = \hat{a}_1 \mu_{\boldsymbol{x}_1}$ e $\hat{a}'_1 = \hat{a}_1 \sigma_{\boldsymbol{x}_1}$.

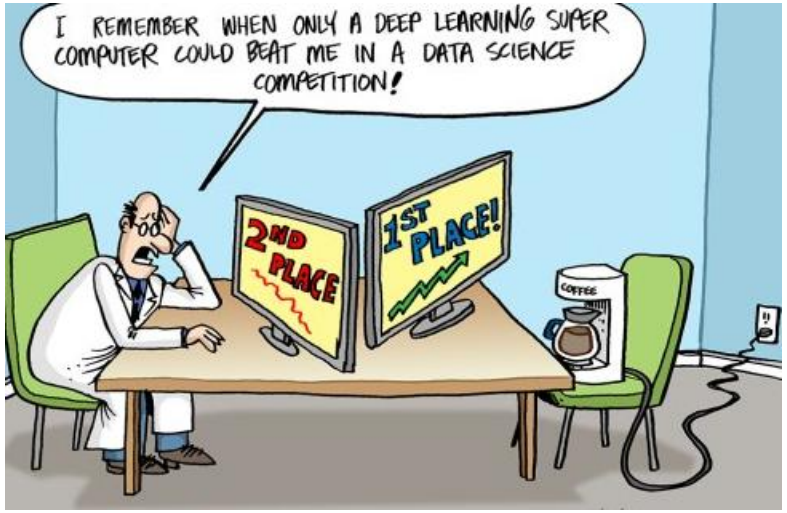
- Note que a padronização de $x_1(n)$ fez com que $\hat{a}_1$ fosse modificado de forma que a função hipótese ainda produza em sua saídas predições condizentes com os valores esperados, $y(n)$.
- Ou seja, mesmo com a padronização dos atributos, a função hipótese ainda fará predições alinhadas aos valores dos rótulos, $y(n)$.
- O mesmo procedimento pode ser diretamente aplicado à normalização e também resultará em mudança dos pesos originais.

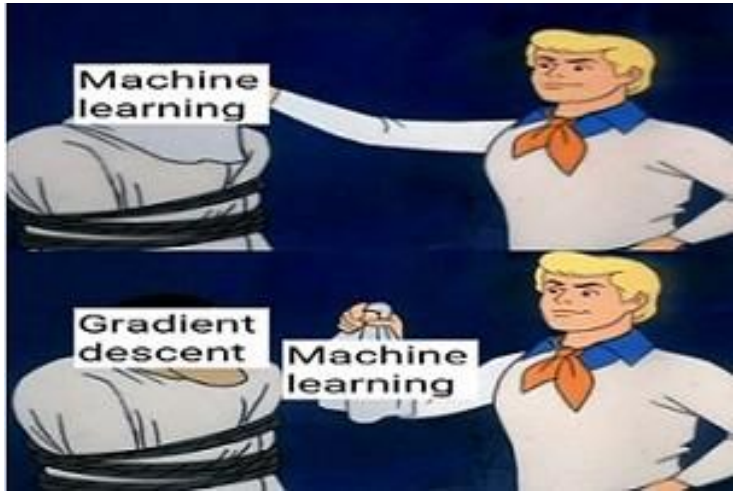


"It's a non-linear pattern with outliers.....but for some reason I'm very happy with the data."



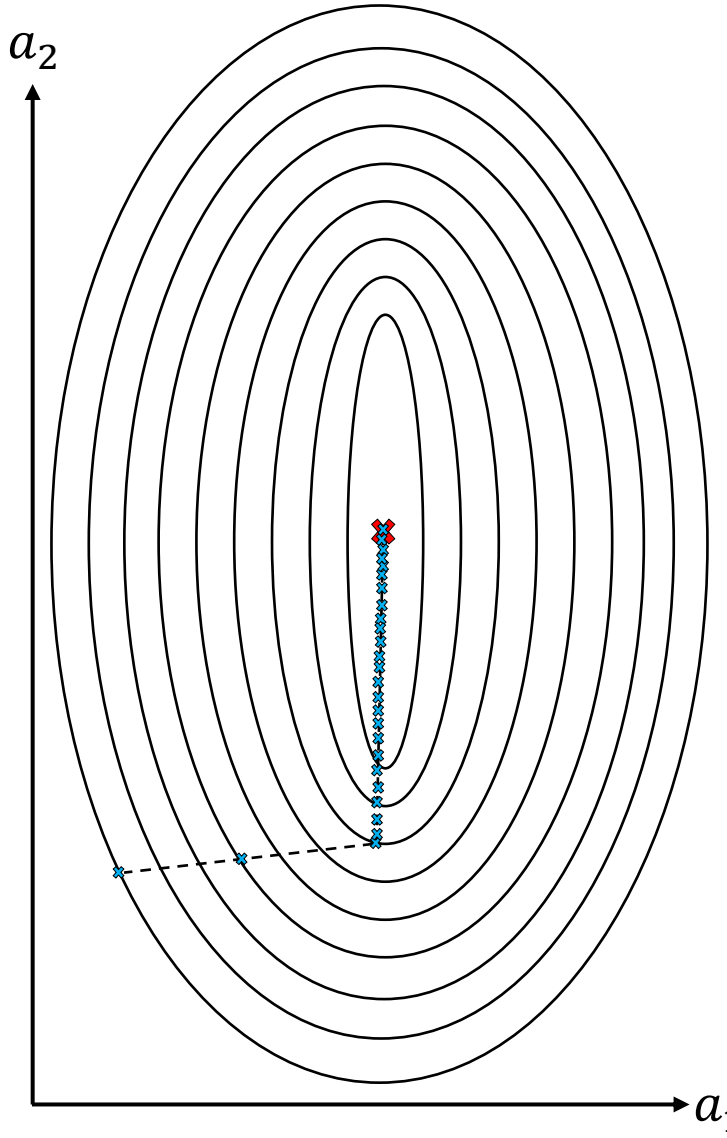


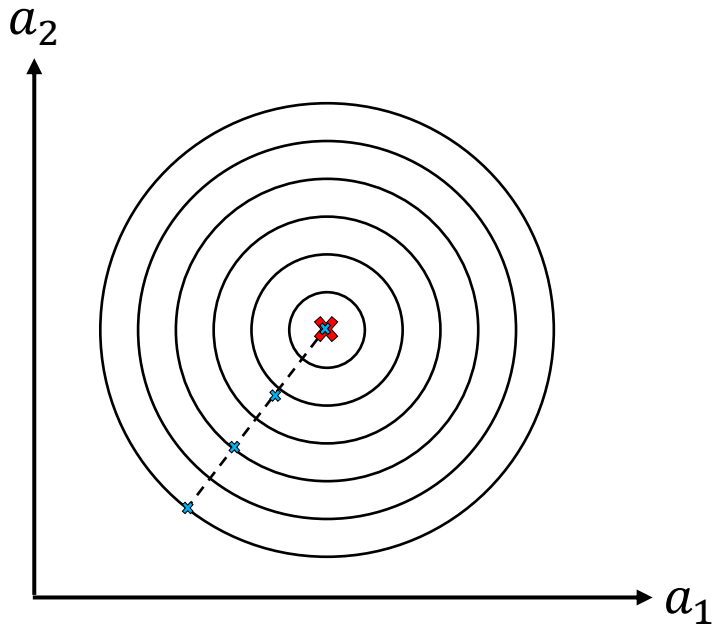






Figuras


a_2
a_1


a_2
a_1